

Reviewer Pack — Minimal Rank of Diophantine Approximation over Boolean Funct...

Entry 1057301fde88 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 23:34:07 UTC

Minimal Rank of Diophantine Approximation over Boolean Functions vs SAT Satisfiability

Entry ID: 1057301fde88 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 23:34:07 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Diophantine Approximation) **Field B** (complexity object): Complexity Theory: SAT Satisfiability

Statement:

{‘sentence_1’: ‘For every instance of a SAT problem with n variables, the minimal index of Diophantine approximation to the truth table entropy of the problem is at most $O(n \log n)$.’, ‘sentence_2’: ‘Equivalently, there exists an absolute constant C such that for all instances of SAT problems with n variables, if the minimal index of Diophantine approximation to the truth table entropy is greater than $Cn \log n$, then the problem is unsatisfiable.’, ‘sentence_3’: ‘No counterexample can have a minimal index less than $O(n \log n)$ while still being satisfiable.’}

Rationale (proposer’s reasoning):

{‘sentence_1’: ‘Diophantine approximation offers a way to measure the complexity of real numbers and their approximations, which may be related to the difficulty of determining the truth value of boolean functions.’, ‘sentence_2’: ‘If this conjecture holds, it would provide a new approach to proving unsatisfiability for SAT problems by studying the approximation properties of their truth tables.’, ‘sentence_3’: ‘This bridge between number theory and complexity could potentially lead to breakthroughs in understanding the hardness of SAT.’}

Taxonomy category: DIOPHANTINE_APPROXIMATION (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 07315c9be2fe8cd2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a SAT problem with n variables, if the minimal index of Diophantine approximation to its truth table entropy is greater than $Cn \log n$ for some absolute constant C or if it is less than $O(n \log n)$, the conjecture is supported; otherwise, it is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n):
        clauses = []
        for _ in range(2**n - 1): # Generate all non-empty subsets of variables
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            clauses.append(clause)
        return clauses

    def truth_table(clauses):
        n = len(clauses[0])
        table = {}
        for assignment in itertools.product([False, True], repeat=n):
            assignment_dict = {i+1: int(assignment[i]) for i in range(n)}
            table[tuple(assignment)] = any(all(lit * assignment_dict[abs(lit)] > 0 for lit in clause) for clause in clauses)
        return table

    def entropy(table):
        n = len(table)
        counts = [sum(table.values()), n - sum(table.values())]
        probabilities = [c / n for c in counts]
        return -sum(p * math.log2(p) if p > 0 else 0 for p in probabilities)

    def diophantine_approximation(entropy):
        # Simplified approximation using continued fractions
        a, b = entropy, 1
        while True:
            q = int(a / b)
            a, b = b, a - q * b
```

```

        if abs(b) == 1:
            return q

n_values = [10, 20, 30, 40]
results = []

for n in n_values:
    for _ in range(7): # Aim for at least 30 instances per seed
        clauses = generate_3cnf(n)
        table = truth_table(clauses)
        entropy_val = entropy(table)
        approx_index = diophantine_approximation(entropy_val)
        results.append((n, approx_index))

if not results:
    return {
        "metric_name": "minimal_diophantine_index",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "no_instances_generated"
    }

n_total = len(results)
approx_indices = [r[1] for r in results]
mean_approx_index = sum(approx_indices) / n_total
std_approx_index = math.sqrt(sum((x - mean_approx_index) ** 2 for x in approx_indices) / n_total)

C = 0.5 # Hypothetical constant, adjust as needed
if any(index > C * n * math.log(n) for n, index in results):
    return {
        "metric_name": "minimal_diophantine_index",
        "metric_value": mean_approx_index,
        "instances_tested": n_total,
        "conjecture_holds": False,
        "counterexample": f"unsatisfiable with  $Cn \log n = \{C * n * \mathbf{log}(n)\}"
    }

return {
    "metric_name": "minimal_diophantine_index",
    "metric_value": mean_approx_index,
    "instances_tested": n_total,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)] # Default to first

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)$ 
```

```

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, r in enumerate(results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='unsatisfiable' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE no_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8a14818e.py", line 100, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8a14818e.py", line 59, in run_trial
    approx_index = diophantine_approximation(entropy_val)
                   ~~~~~^~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8a14818e.py", line 46, in diophantine_approximation
    q = int(a / b)
        ~^~
ZeroDivisionError: float division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a division by zero error before producing data that could confirm or falsify the conjecture. | next: Investigate and fix the division by zero error in the code, then rerun the test with a valid input to determine if the conjecture is supported or falsified.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12729
2	preregistration	ollama_remote	glm4:latest	0	0	5591
3	novelty	ollama_remote	glm4:latest	0	0	5364
4	novelty	ollama_remote	glm4:latest	0	0	5057
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	36447
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9058
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10732
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11776
9	judge	ollama_remote	glm4:latest	0	0	13724

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 110479 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/1057301fde88.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/1057301fde88.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/1057301fde88.tar.gz* (if generated)